

# nginx 로그 데이터 파이프라인 구축

ELK Stack + Apache Kafka 실습 정리

2026년 4월 6일

---

## 목차

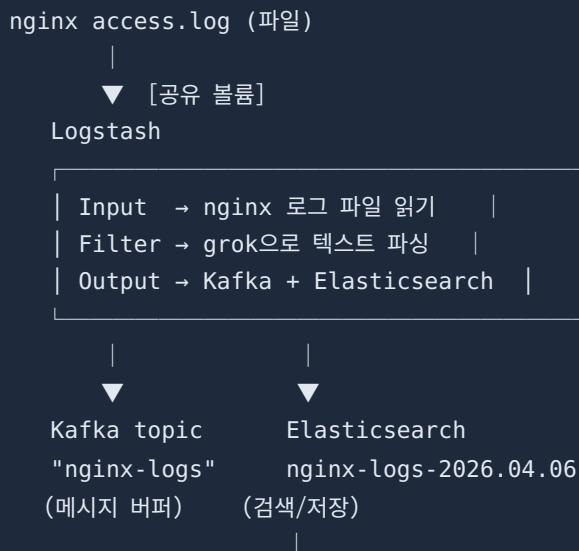
1. 개요 — 왜 이걸 만들었나
2. 전체 아키텍처
3. 핵심 개념 설명
4. 구성 요소별 상세 설명
5. Docker 컨테이너 구성
6. 에러 & 해결 과정
7. Kibana 활용
8. 학습 포인트 정리

## 1. 개요 — 왜 이걸 만들었나

블로그 대시보드에 nginx 접속 로그를 보여주는 위젯을 추가하는 작업이었다. 단순히 로그 파일을 읽어서 보여주는 방식 대신, 실제 데이터 엔지니어링 현장에서 쓰는 파이프라인 구조로 만들어봤다.

이 구조를 **ELK Stack**이라고 부른다. Elasticsearch, Logstash, Kibana의 앞글자를 딴 것이며, 여기에 Kafka를 중간 버퍼로 추가했다.

## 2. 전체 아키텍처





Kibana

(시각화 UI)

+

Next.js 위젯

(블로그 대시보드)

### 3. 핵심 개념 설명

#### 리버스 프록시 (nginx)

클라이언트(브라우저)와 실제 서버(Next.js) 사이에 앉아서 요청을 대신 받아주는 서버다. 외부에서는 nginx만 보이고, 내부 구조는 숨겨진다.

매 요청마다 **access.log**에 기록을 남긴다:

```
172.0.0.1 - - [06/Apr/2026:12:00:00] "GET /api/posts HTTP/2.0" 200 3779 RT=0.023
```

- **172.0.0.1** — 접속자 IP
- **GET /api/posts** — HTTP 메서드와 요청 경로
- **200** — 응답 상태 코드
- **3779** — 응답 본문 크기(bytes)
- **RT=0.023** — 응답 시간(초)

#### HTTP 상태 코드

| 코드      | 의미       | 예시              |
|---------|----------|-----------------|
| 200     | 성공       | 페이지 정상 로딩       |
| 301/302 | 리다이렉트    | HTTP → HTTPS 전환 |
| 404     | 찾을 수 없음  | 없는 URL 요청       |
| 500     | 서버 내부 오류 | 코드 에러           |

#### 메시지 큐 / Kafka

데이터를 생산하는 쪽(Producer)과 소비하는 쪽(Consumer)을 분리해주는 파이프다.

```
Producer (Logstash) → [Kafka topic] → Consumer (Elasticsearch, Next.js...)
```

Kafka 없이 Logstash가 직접 Elasticsearch에 넣으면, Elasticsearch가 느릴 때 Logstash도 막힌다. Kafka가 중간에 있으면 Producer는 계속 밀어 넣고, Consumer는 자기 속도에 맞춰 꺼 내간다.

## KRaft 모드

예전 Kafka는 메타데이터 관리를 위해 ZooKeeper라는 별도 서비스가 필요했다. Kafka 3.x부터는 Raft 합의 알고리즘으로 자체 관리가 가능해져서 ZooKeeper 없이 단독 실행된다. 이를 **KRaft 모드**라고 한다.

## grok 파싱

Logstash에서 비정형 텍스트 로그를 구조화된 JSON으로 변환할 때 쓰는 패턴 매칭 도구다. 정규식의 이름 붙인 버전이라고 보면 된다.

```
# 입력 (텍스트 한 줄)
"172.0.0.1 - - [06/Apr/2026] \"GET /posts HTTP/2.0\" 200 3779 RT=0.023"

# grok 패턴 적용 후 출력 (JSON)
{
  "client_ip": "172.0.0.1",
  "method": "GET",
  "path": "/posts",
  "status": 200,
  "bytes": 3779,
  "response_time": 0.023
}
```

## Elasticsearch

분산 검색 엔진. 로그, 문서 등 대용량 데이터를 저장하고 빠르게 검색할 수 있다. 내부적으로 **역색인 (inverted index)** 구조를 사용해서, 일반 DB보다 텍스트 검색이 훨씬 빠르다.

데이터는 **인덱스** 단위로 저장된다. 우리는 날짜별로 인덱스를 만들었다:

```
nginx-logs-2026.04.06  (오늘 로그)
nginx-logs-2026.04.07  (내일 로그)
nginx-logs-*           (전체 검색할 때 와일드카드)
```

## 4. 구성 요소별 상세 설명

### Logstash 파이프라인 설정 (nginx.conf)

```
input {
  file {
    path => "/var/log/nginx/access.log" # 읽을 파일
    start_position => "beginning"      # 처음부터 읽기
    sincedb_path => "/dev/null"         # 위치 기록 안 함 (개발용)
  }
}

filter {
  grok {
    match => {
      "message" => "%{IPORHOST:client_ip} ... RT=%{NUMBER:response_time:float}"
    }
  }
  if "_grokparsefailure" in [tags] { drop {} } # 파싱 실패 줄 버리기

  date {
    match => ["log_timestamp", "dd/MMM/yyyy:HH:mm:ss Z"]
    target => "@timestamp" # Elasticsearch 표준 시간 필드
  }
}

output {
  kafka {
    bootstrap_servers => "mablog_kafka:9092"
    topic_id => "nginx-logs"
    codec => "json"
  }
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "nginx-logs-%{+YYYY.MM.dd}"
  }
}
```

### Docker 네트워크와 컨테이너 통신

같은 Docker 네트워크에 속한 컨테이너들은 **컨테이너 이름**을 DNS처럼 사용해서 서로 접근할 수 있다.

# 일반 인터넷  
도메인 → DNS 서버 → IP 주소

# Docker 내부 네트워크  
컨테이너 이름 → Docker 내장 DNS → 컨테이너 내부 IP

| 주소                 | 의미                      |
|--------------------|-------------------------|
| mablog_kafka:9092  | Kafka 컨테이너 9092 포트      |
| elasticsearch:9200 | ES 컨테이너 9200 포트 (alias) |
| mablog_nextjs:8080 | Next.js 컨테이너 8080 포트    |

각 포트 번호는 프로그램 기본값이다. Elasticsearch는 9200, Kafka는 9092, Kibana는 5601이 기본이다.

## 5. Docker 컨테이너 구성

### docker-compose.yml 추가 내용

```
# Kafka – KRaft 모드 (ZooKeeper 없음)
mablog_kafka:
  image: apache/kafka:3.7.0
  environment:
    - KAFKA_NODE_ID=1
    - KAFKA_PROCESS_ROLES=broker,controller
    - KAFKA_HEAP_OPTS=-Xms256m -Xmx256m # JVM 힙 256MB로 제한

# Elasticsearch – 단일 노드, 보안 비활성화
mablog_elasticsearch:
  image: elasticsearch:8.13.4
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
  networks:
    mablog_network:
      aliases:
        - elasticsearch # 언더스코어 없는 alias 추가

# Logstash – 파이프라인 파일과 nginx 로그 마운트
mablog_logstash:
  image: logstash:8.13.4
  volumes:
    - ./logstash/logstash.yml:/usr/share/logstash/config/logstash.yml:ro
    - ./logstash/pipeline:/usr/share/logstash/pipeline:ro
    - ./nginx_logs:/var/log/nginx:ro

# Kibana – localhost만 (SSH 터널로 접근)
mablog_kibana:
  image: kibana:8.13.4
  ports:
    - "127.0.0.1:5601:5601"
```

### 메모리 사용량

| 컨테이너 | 역할 | 메모리 |
|------|----|-----|
|------|----|-----|



|                      |             |        |
|----------------------|-------------|--------|
| mablog_kafka         | 메시지 브로커     | ~376MB |
| mablog_elasticsearch | 저장/검색 엔진    | ~978MB |
| mablog_logstash      | 수집/변환 파이프라인 | ~586MB |
| mablog_kibana        | 시각화 UI      | ~511MB |
| 합계                   |             | ~2.4GB |

서버 전체 메모리 23GB 중 약 2.4GB 추가 사용. 기존 컨테이너(MongoDB, Next.js, nginx) 포함해도 총 4GB 수준이라 여유롭다.

## 6. 에러 & 해결 과정

### 에러 1: bitnami/kafka 이미지 없음

#### ERROR

```
failed to resolve reference "docker.io/bitnami/kafka:3.7": not found
```

원인: Bitnami가 Docker Hub 공개 배포를 중단하고 자체 레지스트리로 이전.

해결: 공식 `apache/kafka:3.7.0` 이미지로 교체. 환경변수 접두사도 `KAFKA_CFG` → `KAFKA` 로 변경.

### 에러 2: Elasticsearch 권한 오류

#### ERROR

```
failed to obtain node locks, tried [/usr/share/elasticsearch/data];  
maybe these locations are not writable
```

원인: `elasticsearch_data` 디렉토리가 root 소유로 생성됐는데, ES 컨테이너는 UID 1000으로 실행되어 쓸 수 없음.

해결:

```
sudo chown -R 1000:1000 ./elasticsearch_data
```

개념: Linux 파일 권한에서 각 파일/디렉토리는 소유자(UID)가 있다. Docker 컨테이너 내부의 프로세스도 특정 UID로 실행되며, 호스트의 마운트된 디렉토리에 접근할 때 UID가 맞지 않으면 권한 오류가 난다.

### 에러 3: Logstash 모니터링 연결 실패

#### ERROR

```
Elasticsearch Unreachable: [http://elasticsearch:9200/] name resolution  
failure
```

**원인:** Logstash 8.x 기본 설정에서 xpack 모니터링이 켜져 있고, `elasticsearch` 라는 호스트명으로 연결 시도. 우리 컨테이너는 `mablog_elasticsearch` 라서 찾지 못함.

**해결:** `logstash.yml` 에 명시적으로 비활성화:

```
xpack.monitoring.enabled: false
```

## 에러 4: URI 언더스코어 오류 (핵심)

### ERROR

```
Illegal character in scheme name at index 6: mablog_elasticsearch:9200
```

**원인:** URI 표준(RFC 3986)에서 스킴(scheme) 이름에는 언더스코어(`_`)가 허용되지 않는다.

`http://` 없이 `mablog_elasticsearch:9200` 을 쓰면 Logstash가 `mablog_elasticsearch` 를 스킴으로 파싱하려고 해서 오류가 난다. `http://` 를 붙여도 Java의 URI 파서가 호스트명의 언더스코어를 거부한다.

**해결:** Docker 네트워크 alias로 언더스코어 없는 이름 추가:

```
# docker-compose.yml
mablog_elasticsearch:
  networks:
    mablog_network:
      aliases:
        - elasticsearch # 이 이름으로도 접근 가능

# logstash pipeline에서
hosts => ["http://elasticsearch:9200"] # alias 사용
```

## 에러 5: 볼륨 마운트 변경 미반영

### 현상

nginx에 볼륨 추가했는데 로그 파일이 생기지 않음

**원인:** `docker-compose restart` 는 설정 변경을 읽지 않고 기존 컨테이너를 재시작만 한다. 볼륨 마운트 같은 구조적 변경은 컨테이너 재생성이 필요하다.

**해결:**

```
docker-compose up -d mablog_proxy # 변경된 설정으로 컨테이너 재생성
```

**차이점:** `restart` 는 프로세스를 꺾다 켜다. `up -d` 는 docker-compose.yml과 비교해서 달라진 게 있으면 컨테이너 자체를 새로 만든다.

## 7. Kibana 활용

### SSH 터널로 접근하는 이유

Kibana는 보안상 외부에 직접 노출하지 않도록 `127.0.0.1:5601` 로만 포트를 열었다. 로컬 PC에서 SSH 터널을 열면 로컬의 5601포트가 서버의 5601포트로 연결된다.

```
# 로컬 PC에서 실행
ssh -L 5601:localhost:5601 -i 키파일경로 ubuntu@서버IP -N

# 브라우저에서 접속
http://localhost:5601
```

`-L 5601:localhost:5601` 의 의미: 로컬의 5601포트 → SSH 터널 → 서버의 localhost:5601

### Data View 생성

Kibana에서 Elasticsearch 데이터를 보려면 Data View(인덱스 패턴)를 먼저 만들어야 한다.

- **Index pattern:** `nginx-logs-*` (날짜별 인덱스 전체 포함)
- **Timestamp field:** `@timestamp`

### 실제로 발견한 것

`status: 404` 로 필터링하니 Anthropic의 AI 크롤러(ClaudeBot)가 `robots.txt` 와 `sitemap.xml` 을 요청했다가 404를 받은 기록이 나왔다. 블로그에 이 두 파일이 없었기 때문이다. 구글봇, 빙봇도 똑같이 요청한다.

## 8. 학습 포인트 정리

| 개념            | 핵심 한 줄 요약                                  |
|---------------|--------------------------------------------|
| 리버스 프록시       | 외부 요청을 받아 내부 서버로 전달하는 중간 서버                |
| 메시지 큐(Kafka)  | Producer와 Consumer를 분리해 데이터를 버퍼링하는 파이프     |
| KRaft 모드      | ZooKeeper 없이 Kafka가 자체 메타데이터를 관리하는 방식      |
| grok          | 정규식 기반으로 텍스트 로그를 JSON 필드로 파싱하는 Logstash 기능 |
| Elasticsearch | 역색인 기반 분산 검색 엔진. 대용량 텍스트 검색에 최적화           |
| 역색인           | "단어 → 문서 목록" 형태로 저장해서 텍스트 검색을 빠르게 하는 자료구조  |
| Docker 네트워크   | 같은 네트워크의 컨테이너들은 이름으로 서로 접근 가능              |
| URI 스킴        | http, https, ftp 등 — 언더스코어 사용 불가(RFC 3986) |
| UID 권한        | Docker 컨테이너 내부 프로세스도 UID가 있어서 파일 권한이 적용됨   |
| SSH 터널        | 로컬 포트를 SSH를 통해 원격 서버 포트에 안전하게 연결           |